

From prompt hell to SKILL.md

Frédéric Harper

Senior Developer Relations Manager @ TinyMCE

 fharper



Photo by Alexandre P. Junior on Pexels

Lo siento,
no hablo español.

**I'm going to end this hour by telling you to
let the AI write these.**

So why are we spending an hour on it?

The model can write the file.
It can't know what belongs in it.

Reason one — that part is yours

**You have to be able to tell when what
came back *is wrong*.**

Reason two — that part takes time and back & forth

Same as code review.

You don't need to type it.

You need to **judge it.**

a skill

Repeatable task.

What to do, and when to do it.

**Your agent reads it by itself,
when the task matches.**

A folder.

That's the whole technology.

my-skill/

|— SKILL.md ← required

|— scripts/ optional

|— references/ optional

|— assets/ optional

`name:` pr-description

`description:` Turn a diff into a PR
description in our format. Use when
asked to write or draft a PR.

then just markdown

It loads in three stages.

01

Metadata

name + description only · ~100 tokens · always loaded

01

Metadata

name + description only · ~100 tokens · always loaded

02

Instructions

the full SKILL.md · read when the task matches

01

Metadata

name + description only · ~100 tokens · always loaded

02

Instructions

the full SKILL.md · read when the task matches

03

Resources

bundled scripts and files · loaded only if reached for

Until it fires, the model has only seen the description.

So the description is a trigger.

Same skill, two descriptions

WON'T FIRE

Helps with pull requests.

FIRES

Turn a git diff into a PR description in our team's format. Use when asked to write, draft, or open a PR, or when someone says "what should this PR say".

Write it with the words you'd actually type.

It fires without you.

Including on the days you forgot you wrote it

~100 tokens each. That's why you can have fifty.

It survives review.

A prompt lives in scrollback. A skill lives in a PR.

It's distributable.

The person who joined yesterday gets it for free

It carries what can't be inferred.

The thing you learned the hard way

Doing it once?

Just prompt.

Skills are for the third time.

Let's see one in action

Not just for code

alt-text

A screenshot in, accessible alt text out

travel-info

A trip in Triplt, a briefing on its Notion page out

expensify-report-review

A report in, the lines that don't match the receipts out

10 minutes

workshop

What you need

1. A text editor.
2. Whatever **agent** you already use.

Don't write the skill.

Write the two parts it can't guess.

Two things. Nothing else.

description:

what it does, and when to fire –
in the words you would actually type

What always goes wrong

three things you have hit yourself

The subject — yours.

SKILL.md

name: tinymce-demo

description: Scaffold a runnable TinyMCE demo project for a talk, livestream, blog post, or bug repro. Use when asked for a TinyMCE demo, sample, playground, repro, or "spin up an editor with <plugins>".

TinyMCE demo scaffold

A project folder served on `localhost` and open in a browser within a minute. No npm, no build step...

20 minutes

workshop

Now let it write the whole thing.

Same subject. Give it nothing but the subject.

Now compare it against the things you wrote.

Its description, against yours. Its warnings, against your three.

Generating is the fast path.
Editing is the skill.
It's iterative.

portability

TOOL	WHERE IT GOES	HOW IT FIRES
Claude Code	<code>~/.claude/skills/</code>	automatic
Claude apps	<code>synced/</code>	toggle in settings
Copilot · VS Code	<code>.github/skills/</code>	agent mode
Gemini CLI	<code>~/.gemini/skills/</code>	asks you first
Codex · Cursor · ~40 more	<code>per tool</code>	<code>agentskills.io</code>

One folder, symlinked everywhere

Keep the skills in `~/.agents/skills/`

Symlink it into each tool's own directory

`~/.claude/skills/`

`.github/skills/`

`~/.gemini/skills/`

Plain markdown runs everywhere.

Best practices

Write the description for the matcher.

What it does AND when — in the words you'd actually type

One skill, one job.

A skill that does three things fires cleanly for none of them

Keep SKILL.md under 500 lines.

Push the detail into references/ — it loads only when reached for

Write the gotcha the day you hit it.

That list is the whole value, and it never gets written later

Lessons from writing mine

Tell it what NOT to do.

don't fetch the list of plugins of TinyMCE every run, cost too many tokens

Rules about the output matter as much as the steps.

Length, order, what to cut — that is what makes it reusable

Make it say "I cannot confirm this."

A skill that guesses confidently is worse than no skill

I have a skill that reviews my skills.

skill-optimization — point it at one of mine, it comes back with a numbered list

Nothing you write today is finished.

The gotchas arrive later, one afternoon at a time

in the end

Every time you re-explain something to an agent, that's a skill you haven't written yet.

Resources

THE STANDARD

agentskills.io

the spec, Apache-2.0

agentskills/agentskills

the spec source, plus a validator

agentskills.io/clients

every tool that implements it

GET SKILLS

skills.sh

npx skills add <owner/repo>

anthropics/skills

the best-written examples out there

google/skills

BigQuery, Cloud Run, GKE, Firebase

nvidia/skills

robotics, simulation, CUDA, RAG

CURATED LISTS

composiohq/awesome-claude-skills

the biggest

voltagent/awesome-agent-skills

explicitly cross-tool

travisvn/awesome-claude-skills

well curated



Friday 3:45PM – Gran sala

DEATH TO THE TEXTAREA (AND WHAT TO DO ABOUT IT)

FRÉDÉRIC HARPER





Star Wars or Star Trek? Cast your vote and enter to win!

Stop by [our](#) booth to cast your vote
and [enter to win!](#)







Frédéric Harper

Senior Developer Relations
Manager

